

AA1 — Estudio de caso: integración continua en un proyecto de software

David Alejandro Osorio Martínez

Mayo de 2026

Asignatura: Gestión del software · **Programa:** Ingeniería de software · **Unidad:** 3 — Automatización y entrega continua · **Actividad:** AA1 — Estudio de caso · **Tutor:** Carlos Carrascal Avendaño · **Fecha de entrega:** mayo de 2026

(Verifica el nombre del tutor en la guía de tu sección; si es otro, cámbialo en esta línea antes de entregar.)

Introducción

El caso plantea a una empresa de desarrollo contratada para construir un **sistema de pedidos y reservas en línea para un restaurante**. El equipo avanzó rápido en funcionalidades, pero en **integración y prueba** aparecieron síntomas habituales cuando no hay una práctica sostenida de **integración continua (CI)** ni de verificación automática.

Propósito de este informe: analizar ese contexto con rigor, respondiendo a las cinco preguntas orientadoras; relacionar conceptos de CI con lo aprendido en el curso; y proponer una **solución estructurada** (pasos y beneficios) aplicable a un producto como el prototipo **SIGR** (pedidos, reservas, menú, despliegue), sin perder de vista que el caso habla de un **equipo colaborativo** y de un **cliente** que perdió confianza por entregas inestables.

Marco conceptual

Definición de integración continua

La **integración continua** es una práctica de ingeniería de software en la que cada integración al repositorio compartido (idealmente **varias veces al día**) dispara un proceso **automático y repetible** de verificación: construcción (*build*), pruebas automatizadas, análisis estático u otras comprobaciones acordadas por el equipo. La idea central, difundida en la literatura de la disciplina (p. ej. Fowler, 2006; Duvall *et al.*, 2007), no es “tener un servidor”, sino **acortar el ciclo de feedback** entre el cambio y la evidencia de que el sistema sigue siendo **coherente en conjunto**.

Herramientas comunes (al menos tres)

1. **GitHub Actions** — definición de *workflows* en YAML disparados por eventos del repositorio (*push*, *pull request*, cron); encaja cuando el código ya vive en GitHub y se busca CI sin operar infraestructura propia desde el día uno.
2. **GitLab CI/CD** — *pipeline* en el mismo ecosistema que el control de versiones; útil para unificar código, revisiones y jobs.
3. **Jenkins** — automatización muy flexible con plugins; común en organizaciones con servidores propios o integraciones heterogéneas.

Complementarias frecuentes: **Azure Pipelines**, **CircleCI**, **SonarQube** o **ESLint** para calidad estática, y **Docker** para homogeneizar el entorno de *build* y prueba.

Relación entre CI y metodologías ágiles

Las metodologías ágiles buscan **entrega incremental** y aprendizaje temprano. La CI es el **soporte técnico** que evita confundir “historia terminada en el tablero” con “incremento realmente integrado”. Sin integración frecuente verificada, el *incremento* queda en ramas aisladas. Prácticas como **revisiones en *pull request***, ramas de vida corta o integración frecuente a la línea principal fortalecen Scrum o Kanban porque el tablero refleja trabajo **ya probado en conjunto**, no solo en una laptop.

Desarrollo del análisis

Síntesis del problema planteado en el estudio de caso

Para cumplir con el criterio de **análisis y resolución del problema**, conviene dejar explícito qué falló y por qué duele:

Situación descrita en el caso	Efecto en el proyecto
Ramas separadas sin integración frecuente	Conflictos de fusión difíciles, retrabajo y riesgo de romper funcionalidad ajena al integrar tarde.
Pocas pruebas automatizadas aplicadas con regularidad	Errores críticos llegan a etapas avanzadas; el costo de corrección sube.
Versiones inestables entregadas al cliente	Cae la confianza; el negocio percibe riesgo en lugar de avance.
Tiempo perdido en corregir defectos evitables	Menos capacidad para nuevas funciones; el equipo se vuelve reactivo.

La **solución propuesta** (no única, pero coherente con el caso) es adoptar **CI** como hábito de equipo: integración frecuente a una rama principal, *pipeline* automático que falle rápido ante regresiones, y criterios claros de calidad antes de fusionar. Lo siguiente desarrolla esa línea conectándola con las preguntas orientadoras.

Pregunta 1. ¿Por qué es importante aplicar integración continua desde las primeras fases del desarrollo de software?

Porque el costo de un defecto y de un conflicto de integración **no crece de forma lineal**: cuanto más grande es el conjunto de cambios acumulados sin verificar, más difícil es aislar la causa y más caro es arreglarlo. En etapas tempranas el sistema todavía es **acotado**; integrar y probar seguido mantiene un “radio de explosión” pequeño: cada *merge* aporta un cambio entendible y una verificación automática que dice *sigue funcionando el todo*.

Además, la CI empuja a definir temprano **cómo se construye** el producto y **cómo se prueba** lo mínimo indispensable, lo que reduce la dependencia de “el compañero que sabe cómo armar la demo”. En el restaurante, eso se traduce en que pedidos, reservas y administración no vivan como islas en ramas paralelas, sino que converjan en una línea principal **constantemente verificada**.

Pregunta 2. ¿Qué herramientas y prácticas se utilizan comúnmente para implementar la integración continua? Mencione al menos tres.

Tres herramientas ya citadas en el marco conceptual: **GitHub Actions**, **GitLab CI/CD** y **Jenkins**.

Prácticas que suelen acompañar a la CI y que **justifican** su uso:

- **Repositorio Git** con política explícita de fusión (por ejemplo, revisión obligatoria en *pull request*).
 - **Build reproducible** (mismos comandos en local y en CI; idealmente contenedor o script único).
 - **Pruebas automatizadas** en pirámide razonable: muchas unitarias, algunas de integración o contrato, pocas *end-to-end* pero bien elegidas.
 - **Umbral de calidad acordado** (*definition of done*): “no fusionar si el *pipeline* está rojo”.
 - **Artefactos trazables** (lo que se despliega queda ligado a un commit y a un *run* exitoso).
-

Pregunta 3. ¿Qué consecuencias puede tener la ausencia de integración continua en proyectos colaborativos?

Sin CI (y sin integración frecuente real), aparecen, entre otras:

- **Conflictos de fusión** masivos y propensos a error humano al resolverlos.
- **Regresiones tardías** sin un commit claro que explique el cambio de comportamiento.
- **Cultura de “en mi máquina sí”** que erosiona la confiabilidad del entregable.

- **Incertidumbre para el cliente** y para el negocio: las demos se convierten en apuestas.
- **Deuda técnica** en scripts, configuración y pruebas que “luego se arreglan” y casi nunca se priorizan.

En el caso del restaurante, esas consecuencias ya se materializaron en **desconfianza** y en tiempo perdido corrigiendo lo que un *pipeline* mínimo podría haber detectado antes.

Pregunta 4. ¿Cómo influye la integración continua en la calidad del producto final y la satisfacción del cliente?

La CI mejora la **calidad interna** porque introduce un **filtro automático** antes de que el cambio forme parte de la línea oficial: compila, prueba, y opcionalmente analiza. Eso reduce la tasa de defectos que llegan a integración manual o a producción.

La **calidad percibida** por el cliente mejora cuando las versiones que ve en *staging* o en demos provienen de un proceso **reproducible** (mismo binario o mismo artefacto que pasó pruebas), no de un empaquetado manual a última hora. La **satisfacción** sube cuando las conversaciones pasan de “arreglamos esto para la reunión” a “esto ya está integrado y verificado; hablemos de la siguiente prioridad del negocio”.

Pregunta 5. Si el equipo implementara integración continua en este proyecto del sistema del restaurante, ¿qué pasos debería seguir y qué beneficios concretos obtendría?

Pasos recomendados (orden práctico)

1. **Congelar una política de ramas:** main como línea integrada; ramas de función cortas; fusión solo con revisión y *pipeline* verde.
2. **Definir el *build* mínimo reproducible:** por ejemplo, en un front estático, validación de dependencias, *lint* y pruebas cuando existan; con backend futuro, **build + test** del servicio.
3. **Elegir la plataforma de CI** acorde al repositorio (p. ej. **GitHub Actions** si el código está en GitHub).
4. **Crear el primer *workflow* pequeño** que corra en cada *push* y en cada PR: instalar, construir, ejecutar pruebas.
5. **Crece la suite con criterio:** reglas de totales en pedidos, validaciones de reservas, contratos de API cuando existan REST; pocas *e2e* sobre flujos críticos (checkout, reserva).
6. **Publicar a *staging*** cuando main esté verde, con URL estable para el cliente.
7. **Medir el *pipeline*:** tiempo de ejecución y *flakes*; una suite lenta o intermitente se sabotea sola.

Beneficios concretos para el sistema del restaurante

- **Menos sorpresas al integrar** módulos de pedidos, reservas y administración.
- **Detección temprana** de regresiones antes de comprometer fechas con el restaurante.
- **Trazabilidad** entre versión mostrada al cliente y commit verificado.
- **Recuperación gradual de confianza** del cliente al ver estabilidad repetible.
- **Base para entrega continua (CD)** cuando el equipo y el negocio estén listos para automatizar despliegues con salvaguardas.

Esquema de *pipeline* (referencia visual en texto)

evento (push / PR) → checkout → instalación de dependencias → lint (opcional)
→ build → pruebas automatizadas → (opcional) artefacto / despliegue a staging

Roles típicos: desarrolladores (mantener pruebas y estándares), quien apoye DevOps/QA (mantener el *workflow*), *product owner* o cliente (validar en *staging*). **Riesgos mitigados:** integración tardía, regresiones no detectadas y dependencia de empaquetados manuales.

Conclusión

El estudio de caso muestra un patrón claro: **velocidad sin integración disciplinada** termina en conflictos, defectos tardíos y pérdida de confianza del cliente. La **integración continua** no sustituye el buen diseño ni la comunicación con el negocio, pero sí **obliga** a que el trabajo colaborativo sea verificable en pequeños incrementos y que el software permanezca **integrado** en una línea principal, no disperso en ramas paralelas.

Como **aprendizajes** principales se destacan: (1) la CI debe plantearse **desde el inicio**, no como remiendo al final; (2) herramientas como **GitHub Actions**, **GitLab CI** o **Jenkins** son vehículos, pero el núcleo es la **práctica** de integrar y probar con frecuencia; (3) sin CI, los proyectos colaborativos sufren costos visibles en **merge**, regresiones y percepción del cliente; (4) con CI, la calidad del producto y la satisfacción del cliente mejoran porque lo entregable es **más predecible**; y (5) en el sistema del restaurante, una adopción por etapas —política de ramas, *pipeline* mínimo, pruebas incrementales, *staging*— es una **solución argumentada** y alineada con los fundamentos teóricos del curso.

Referencias (lectura complementaria)

- Fowler, M. (2006). *Continuous Integration*. martinowler.com — artículo de referencia sobre la práctica y sus fundamentos.
- Duvall, P. *et al.* (2007). *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley.